Exam Proctor - Full Documentation

Project documentation | Version 1 | October 22, 2026

1. What this system does

Exam Proctor is a web application for online tests. While a student answers questions, the system watches the student's webcam, decides whether the student appears to be cheating, shows a warning on screen, and saves the time of every suspicious moment together with a short video clip. A reviewer later opens a review page and watches only the flagged clips instead of the whole recording.

The main focus is **eye and head tracking**: the system detects when a student looks away from the screen. It also detects a missing face, more than one person, tab switching, leaving fullscreen, and copy/paste/right-click.

Important: a flag is evidence for a human reviewer, not proof of cheating. People look away to think. Never fail a student automatically based on flags alone.

2. How it works (architecture)

1. The student opens `/exam`, enters a name and clicks **I agree - start exam**.
2. The browser asks for camera permission, enters fullscreen, and loads the face-tracking model (it was already pre-loading while the student typed).
3. A 3-second **calibration** records where the student's eyes and head point when they look at the screen normally.
4. The exam is shown. About 10 times per second the browser analyses the camera image.
5. When a suspicious condition lasts long enough, an **event** opens, the student sees a warning, and the system keeps video segments around that moment.
6. When the event ends, the clip segments are uploaded to storage and the event details (type, time, duration, clip links) are saved.
7. On **Submit exam**, the system waits for all uploads to finish and shows a **Review** button.
8. The reviewer opens `/review`, sees flags grouped by student, and plays the clips.

Why the detection runs in the browser

Vercel runs *serverless functions*: short-lived programs with time and request-size limits. They cannot hold a live video stream and run face analysis on it. So the analysis runs on the student's own computer, using JavaScript and WebAssembly. This gives three benefits: no video is streamed to a server all the time, there is no waiting for a network round-trip, and the cost on Vercel stays very low. The server is only used to hand out upload permission and to store and list results.

Technology used

Part	Technology
Web framework	Next.js 14 (App Router) and React 18
Face / eye / head tracking	MediaPipe Face Landmarker (`@mediapipe/tasks-vision` 0.10.14), runs in the browser
Video clips	Browser `MediaRecorder` API
Storage	Vercel Blob (public store) in production; a local `.data` folder when testing locally
Hosting	Vercel, deployed from GitHub

3. Research basis

Three papers guided the design.

Gupta, Sharma and Tripathi - "Tracking Eye Movement Using Facial Keypoints and Image Processing" (2023). This paper has the same goal: detect a candidate looking away during an online exam. It uses dlib face detection (HOG + linear SVM), 68 facial landmarks, thresholding and morphology to find the pupils, and an alert rule based on pupil distance to the eye corners. Its own limitations: it needs manual threshold calibration, fails in dim or uneven lighting, performs poorly with glasses, and fails when the candidate sits far from the camera. It is also a Python/OpenCV pipeline that does not fit Vercel. **What I took:** the idea of a first-layer proctor that alerts when attention leaves the screen, and a time threshold before alerting. **What I changed:** the pupil-thresholding method is replaced by MediaPipe, and the manual slider is replaced by automatic calibration.

Belmonte et al. - "Impact of Facial Landmark Localization on Facial Expression Recognition" (IEEE Transactions on Affective Computing). It shows that landmark accuracy does not automatically translate into good results for the task built on top, that head movements out of the image plane (yaw, pitch, diagonal) are the hardest cases for landmark detectors, and that unstable landmarks over time cause errors. **What I took:** do not rely on eyes alone, so head yaw and pitch are used together with eye direction; and require a condition to persist over time before flagging, which filters jitter.

Iskra and Tomc - "Eye-tracking analysis of face observing and face recognition" (2016). It reports that normal fixations last roughly 200-350 ms, that people naturally move their gaze between face areas, and that eye trackers lose data because of glasses, makeup and ambient light (the authors discarded recordings below 90% valid samples). **What I took:** short glances are normal, so a look-away must last 1.5 seconds before it counts; and data loss is handled explicitly by the separate "face not visible" flag.

4. Project structure

```
proctor/
|-- package.json          dependencies and scripts
|-- next.config.js        Next.js settings
|-- .gitignore            files Git must not upload
|-- README.md             short run notes
|-- lib/
|   |-- proctor.js        the proctoring engine (browser side)
|   `-- store.js          storage helpers (server side)
`-- app/
    |-- layout.js         page shell
    |-- globals.css       all styling
    |-- page.js           home page with two links
    |-- exam/page.js      student exam page
    |-- review/page.js    reviewer page
    `-- api/
        |-- upload/route.js   gives permission to upload a clip
        |-- event/route.js    saves one event
        |-- events/route.js   lists all events
        `-- local/route.js    local-only clip storage
```

5. File-by-file explanation

5.1 package.json

Lists the runtime dependencies: `next` 14.2.15, `react` and `react-dom` 18.3.1, `@vercel/blob` (storage client and server helpers) and `@mediapipe/tasks-vision` 0.10.14 (face tracking). The scripts are `dev` (development server), `build` (production build, used by Vercel) and `start`.

5.2 next.config.js

Sets `reactStrictMode` to `false`. In development, strict mode mounts components twice, which would start the camera and model twice. Turning it off avoids that.

5.3 .gitignore

Tells Git to skip `node_modules`, `.next` (build output), `.data` (local test data) and `.env*` (secrets). These must never be uploaded to GitHub.

5.4 lib/proctor.js - the proctoring engine

This is the most important file. It runs only in the browser.

Configuration constants (top of file)

Constant	Value	Meaning
`TH.h`	0.5	Horizontal eye-direction deviation allowed (scale 0-1)
`TH.v`	0.55	Vertical eye-direction deviation allowed (scale 0-1)
`TH.yaw`	25 deg	Head turn left/right allowed
`TH.pitch`	20 deg	Head tilt up/down allowed
`HOLD.gaze_away`	1500 ms	How long looking away must last before a flag
`HOLD.no_face`	2500 ms	How long no face must be seen before a flag
`HOLD.multiple_faces`	1500 ms	How long a second face must be seen before a flag
`GRACE`	700 ms	How long the condition must be gone before the event closes
`SEG`	4000 ms	Length of each recorded video segment
`MAXSEGS`	8	Maximum segments per event clip (about 32 seconds)

The thresholds `TH` are *deviations from the student's own calibrated baseline*, not absolute angles. This is why different seats, camera heights and screen sizes still work.

loadModel() - Loads MediaPipe once and caches the result. It downloads the WebAssembly runtime from jsDelivr and the Face Landmarker model file from Google's storage. It tries the GPU first and falls back to the CPU. The model is configured to detect up to 2 faces and to output *blendshapes* (numbers describing facial movements such as eye direction) and the *facial transformation matrix* (the 3D head pose). The exam page calls this as soon as it opens, so the download happens while the student types their name.

readSignals(result) - Converts one MediaPipe result into four numbers plus a face count:

- `faces`: how many faces were found.
- `h` (horizontal gaze): `((eyeLookOutRight + eyeLookInLeft) - (eyeLookOutLeft + eyeLookInRight)) / 2`. It is positive when the eyes point to one side and negative for the other.
- `v` (vertical gaze): average of the two `eyeLookUp` values minus the average of the two `eyeLookDown` values.
- `yaw` and `pitch`: computed from the head-pose matrix (`yaw = atan2(m[8], m[10])`, `pitch = asin(-m[9])`), converted to degrees.

Because only the *absolute difference from the baseline* is used later, the sign convention does not matter.

Class Proctor - One instance per exam session.

- `constructor` stores the video element, camera stream, session id, student name and the `onWarn` callback, and creates empty state: baseline, recording buffer (`ring`), open events (`ev`), per-condition timers (`cond`), upload promises (`pending`), warning count and start time `t0`.
- `start()` loads the model, picks the best supported video format (WebM VP8, then WebM, then MP4 for Safari), starts the segment recorder, attaches browser event listeners and starts the detection loop.

- `calibrate(ms)` collects signals for 3 seconds while exactly one face is visible. It needs at least 8 valid samples, otherwise it throws "Face not detected". The **median** of each signal becomes the baseline (the median ignores blinks and outliers).
- `loop()` runs every 90 ms (about 10 checks per second). It only analyses when the video has produced a new frame, then calls `step()`. Ten checks per second is enough because the rules are measured in seconds.
- `step(signals)` is the decision step. During calibration it only collects samples. Otherwise it computes `away`: true if exactly one face is present and any of eye-H, eye-V, yaw or pitch deviates from the baseline by more than its threshold. It then updates three trackers: `no_face`, `multiple_faces` and `gaze_away`.
- `track(type, on, now)` is the hold/grace state machine (see section 6).
- `flag(type, on)` handles instant conditions that need no hold time (tab hidden, window blur, left fullscreen).
- `open(type, start)` creates an event: a new id, the start time, and the recording buffer as pre-roll. It increases the warning count and calls `onWarn` so the student sees the message.
- `close(type, end)` marks the event as ending. The clip is finalized when the next video segment completes (post-roll).
- `cycle()` is the segment recorder (see section 7).
- `onSeg(blob)` is called whenever a segment completes. It keeps the latest segment as pre-roll, appends it to every open event, and finalizes events that are closing or have reached `MAXSEGS`.
- `finish(event)` removes the event from the open list, uploads all its segments in parallel via `up()`, then saves the event record with `post()`. The promise is stored in `pending` so that submit can wait for it.
- `up(blob, name)` first tries the Vercel Blob client upload through `/api/upload`. If that fails (for example, no token during local testing), it falls back to `/api/local`. If both fail it returns null, and the event is saved with fewer or no clips.
- `post(event)` sends the event JSON to `/api/event` with `keepalive`, so it still goes out if the page is closing.
- `bind()` registers browser listeners: `visibilitychange` (tab switch), `blur` and `focus` (window loses focus, flagged only after 2 seconds), `fullscreenchange` (left fullscreen), and `copy`, `paste`, `cut`, `contextmenu` (blocked, and logged as a `clipboard` event with no clip).
- `stop()` stops detection, removes all listeners, finalizes any open events immediately and waits for all uploads. Listeners are removed *before* the exam page leaves fullscreen, so submitting does not create a false flag.

5.5 lib/store.js - storage helper (server only)

- `useBlob` is true when the environment variable `BLOB_READ_WRITE_TOKEN` exists. On Vercel with a connected store, it is true.
- `clean(string)` removes every character except letters, digits, underscore, dot and hyphen. It is used on all file names so nobody can write to unexpected paths.
- `saveLocal(path, data)` writes a file inside the `.data` folder.
- `readLocalEvents()` reads every JSON file in `.data/events` and returns the list.

The local mode exists so you can test on your own computer without any Vercel account setup.

5.6 app/api/upload/route.js

Handles the secure client-upload handshake of `@vercel/blob`. The browser asks for permission; this route replies with a short-lived token. It only allows `video/webm` and `video/mp4`, a maximum of 15 MB per file, and adds a random suffix to file names. This design matters because Vercel functions accept request bodies of only about 4.5 MB, while the browser uploads clips directly to Blob storage, avoiding that limit.

5.7 app/api/event/route.js

Receives one event as JSON and stores it as `events/<session>-<id>.json`: in Vercel Blob when `useBlob` is true, otherwise in `.data/events`. Returns `{ok:true}` or an error message.

5.8 app/api/events/route.js

Returns every saved event as a JSON array sorted by start time. With Blob it lists the `events/` prefix (up to 1000 files), downloads each JSON and combines them. Locally it reads `.data/events`. Errors are returned as `{error: message}` with status 500. **This route has no password**, so anyone who knows the address can read the data.

5.9 app/api/local/route.js

Local-testing clip storage. `POST ?name=...` saves a clip into `.data/clips`; `GET ?name=...` serves it back. It refuses to work (403) when a Blob token exists or when running on Vercel, because Vercel's file system is read-only.

5.10 app/layout.js, app/globals.css, app/page.js

`layout.js` is the HTML shell and loads the stylesheet. `globals.css` holds all styles, including automatic light and dark colours, the small mirrored camera preview in the bottom-right corner, the red warning banner, the full-screen calibration overlay and the review table. `page.js` is the home page with links to `/exam` and `/review`.

5.11 app/exam/page.js - the student page

A client component with five phases: `intro` (name and consent), `calibrating` (3-second overlay), `exam` (questions), `saving` (waiting for uploads) and `done` (Submitted screen with the **Review** button).

On load it starts `loadModel()` in the background. When the student clicks start it requests fullscreen and the camera (640x480 at 15 fps, which keeps processing and video size small), creates the `Proctor`, starts it, calibrates, and shows the questions. The warning banner appears for 4 seconds each time `onWarn` fires and shows the running flag count. Submit calls `proctor.stop()`, releases the camera and leaves fullscreen.

The three sample questions are hard-coded in the `QS` array at the top. **The selected answers are kept only in the page's memory and are not saved anywhere** (see limitations).

5.12 app/review/page.js - the reviewer page

Loads events automatically from `/api/events`, groups them by session (one block per student), and shows a table: exam time (mm:ss from the exam start plus the clock time), type, length and a video player. A student's header shows the total number of flags and total flagged seconds. Because a clip is made of several 4-second segments, the `Player` plays them one after another and loops; a label shows "part 1/3". The **Refresh** button reloads the list.

6. Detection logic in detail

Events and their meaning

Type	Trigger	Hold time
`gaze_away`	One face, and eyes or head deviate beyond thresholds from baseline	1.5 s
`no_face`	No face found	2.5 s
`multiple_faces`	Two or more faces found	1.5 s
`tab_hidden`	The tab is hidden or the student switches tabs	None
`window_blur`	The exam window lost focus	2 s delay
`fullscreen_exit`	The student left fullscreen	None

Type	Trigger	Hold time
`clipboard`	Copy, paste, cut or right-click attempted (blocked)	None, logged without a clip

The hold / grace state machine

For every continuous condition the engine remembers when it became true (`since`) and when it stopped (`off`).

1. While the condition is true and the event is not open, the engine waits. Once the condition has been true for the hold time, the event opens, and its **start time is set to when the condition began** (not when the hold ended).
2. When the condition becomes false, the timer resets. If an event is open, the engine waits for the 700 ms grace period. If the condition is still false, the event closes and its **end time is when the condition stopped**.
3. If the condition returns during the grace period, the same event continues. This stops a blink or a one-frame detection error from splitting one event into many.

The hold time removes normal glances (research says fixations last a fraction of a second); the grace time removes flicker.

7. Clip recording logic

A video file written by `MediaRecorder` is only playable if it contains its header. So the system records **separate 4-second segments**, each a complete playable file, instead of one long recording.

1. `cycle()` starts a recorder, and after 4 seconds stops it and immediately starts the next one.
2. Each finished segment goes to `onSeg()`. The newest segment is kept as the *pre-roll buffer*.
3. When an event opens, the buffer (up to 4 seconds before the flag) is copied into the event's list.
4. Every segment that finishes while the event is open is appended.
5. When the event has closed and one more segment completes, the clip is final: **up to 4 s before, the event itself, up to 4 s after**. If the event lasts very long, it is cut at 8 segments (about 32 seconds) and, if the condition continues, a new event starts.
6. The reviewer page plays the segments in order.

There may be a few milliseconds of gap between segments when the recorder restarts. The video bitrate is 350 kbps, so a 12-second clip is roughly 500 KB.

8. Data stored for each event

Field	Meaning
`id`	Unique id of the event
`sessionId`	Unique id of this student's exam session
`student`	Name typed by the student
`type`	One of the event types in section 6
`start`, `end`	Time in milliseconds since 1970 (Unix time)
`durationMs`	Event length in milliseconds
`offsetSec`	Seconds from exam start to the event, shown as mm:ss
`clips`	List of video segment URLs (empty for `clipboard`)

In Blob storage the files are `events/...json` and `clips/<session>/<event>-<n>.webm`.

9. How to run on your computer

Requirements: Node.js 18 or newer (check with `node -v`) and Chrome or Edge with a webcam.

1. Unzip the project and open a terminal in the folder that contains `package.json`.
2. Run `npm install` (once).
3. Run `npm run dev`.
4. Open `http://localhost:3000/exam`, allow the camera and complete the test. The camera works on `localhost` without HTTPS.
5. Look away from the screen for about 2 seconds, or switch tabs, to create a flag.
6. Click **Submit exam**, wait for the Submitted screen, then click **Review**.

Without a Blob token, flags and clips are stored in the `.data` folder inside the project. Delete that folder to clear test data. After changing any file, stop the server with `Ctrl+C` and start it again.

10. How to deploy (GitHub and Vercel)

Upload to GitHub

1. Create an empty repository on github.com (for example `exam-proctor`).
2. In the project folder run: `git init`, `git add .`, `git commit -m "first commit"`, `git branch -M main`, `git remote add origin <your repository address>`, `git push -u origin main`. Use `git add .` (with the dot) so that all files are included, not only the README.
3. Check that `package.json`, `app` and `lib` appear at the top level of the repository.

Deploy on Vercel

1. Sign in at vercel.com with GitHub, choose **Add New -> Project**, import the repository and click **Deploy**. The Next.js settings are detected automatically.
2. Open the project's **Storage** tab, create a **Blob** store, set access to **Public**, and connect it to the project for all environments. This adds `BLOB_READ_WRITE_TOKEN`.
3. Go to **Deployments**, open the menu on the latest deployment and choose **Redeploy**. A new deployment is required for the token to take effect.
4. Students use `https://<your-project>.vercel.app/exam`; reviewers use `/review`.

To update the live site later, change files, then run `git add .`, `git commit -m "message"`, `git push`. Vercel redeploys automatically.

Why the store must be Public: the code uploads with public access. A store created as Private rejects those uploads.

11. Tuning guide

All values are at the top of `lib/proctor.js`.

Problem	Change
Too many "looking away" flags	Raise `TH.h`, `TH.v`, `TH.yaw` or `TH.pitch`, or raise `HOLD.gaze_away`
Real looking away is missed	Lower the thresholds or the hold time
Clips are too short or too long	Change `SEG` and `MAXSEGS`

Problem	Change
Video files too large	Lower `videoBitsPerSecond` in `cycle()`

Change one value at a time and test with several people, with and without glasses, and in different lighting.

12. Limitations and known issues

- **Answers are not saved.** The exam page keeps answers only in memory. For real use you must send them to a database or an API.
- **No authentication.** The review page and `/api/events` are open to anyone with the link. There is no student login.
- **Public storage.** Blob links are unguessable but technically public. For real exams with sensitive video, a private store with server-checked access is advisable; that needs code changes.
- **Thresholds are untested starting values.** They must be tuned with real users before any real exam.
- **Glasses, dim light, uneven light or sitting far from the camera** reduce accuracy; this matches the limits reported in the papers.
- **Browser-side checks can be bypassed** by a determined student (for example, a second device or a virtual camera).
- **No audio** is recorded, so talking to someone off-camera cannot be detected.
- **No phone or object detection.**
- **Refreshing the exam page** ends the session; there is no resume.
- **Silent upload failures:** if a clip upload fails, the event is still saved, possibly with no clip.
- **The in-progress segment** is lost for events still open at the moment of submit (up to 4 seconds).
- **window_blur** can fire when the student clicks the browser's own interface or developer tools.
- **Review list limit:** `/api/events` reads up to 1000 event files.
- **Privacy and consent:** recording students is regulated in many places. The intro screen has a short notice, but you must check your local legal requirements and your institution's policy.

13. Troubleshooting

Symptom	Likely cause and fix
Camera does not start	Use Chrome or Edge, click Allow, close other apps using the camera, and on Vercel make sure the address begins with `https://`
"Face not detected" at calibration	Improve lighting, face the camera, move closer, then try again
Review page says "unauthorized"	You are running an old version of `app/api/events/route.js` that still had a password check; use the current file
Review page shows a token error	The Blob store is missing, not connected, or you have not redeployed since connecting it
Uploads rejected	The Blob store was created as Private; recreate it as Public
Flags appear but clips say "no clip"	Open the browser console (F12) during submit and check the red errors; confirm the token and store
Changes do not appear locally	Stop and restart `npm run dev`
Only README reached GitHub	You ran `git add README.md`; run `git add .`, then commit and push

Symptom	Likely cause and fix
Build fails on Vercel	Open the failed deployment, copy the last red lines of the log and check that `package.json` is at the repository top level

14. Possible next steps

- Student login and an exam/question bank with saved answers.
- A password or login for reviewers, and a private store.
- Audio recording and detection of a second voice.
- Phone or object detection.
- A reviewer workflow: mark flags as confirmed or dismissed, add notes, and export a report.
- A risk score per student based on the number and length of flags.
- Reconnect and resume support.